CLI

Using Agent in CLI

Modes

The CLI supports the same modes as the editor. Switch modes using slash commands or the `--mode` flag.

Plan mode

Use Plan mode to design your approach before coding. The agent asks clarifying questions to refine your plan.

- Press `Shift+Tab` to rotate to Plan mode
- Use `/plan` to switch to Plan mode
- Start with `--plan` or `--mode=plan` flag

Ask mode

Use Ask mode to explore code without making changes. The agent searches your codebase and provides answers without editing files.

- Use `/ask` to switch to Ask mode
- Start with `--mode=ask` flag

Prompting

Stating intent clearly is recommended for the best results. For example, you can use the prompt "do not write any code" to ensure that the agent won't edit any files. This is generally helpful when planning tasks before implementing them.

Agent has tools for file operations, searching, running shell commands, and web access.

MCP

Agent supports MCP (Model Context Protocol) for extended functionality and integrations. The CLI will automatically detect and respect your `mcp.json` configuration file, enabling the same MCP servers and tools that you've configured for the editor.

ACP

Agent also supports ACP (Agent Client Protocol) for custom client integrations. Use `agent acp` to run Cursor CLI as an ACP server over `stdio` with JSON-RPC messaging.

Rules

The CLI agent supports the same rules system as the editor. You can create rules in the `.cursor/rules` directory to provide context and guidance to the agent. These rules will be automatically loaded and applied based on their configuration, allowing you to customize the agent's behavior for different parts of your project or specific file types.

> The CLI also reads `AGENTS.md` and `CLAUDE.md` at the project root (if present) and applies them as rules alongside `.cursor/rules`.

Working with Agent

Navigation

Previous messages can be accessed using arrow up (`↑`) or `Ctrl-p`, where you can cycle through them.

Previous messages can be accessed using arrow up (`Arrow Up`) where you can cycle through them.

Input shortcuts

- `Shift+Tab` — Rotate between modes (Agent, Plan, Ask)
- `Shift+Enter` — Insert a newline instead of submitting, making it easier to write multi-line prompts.
- `Ctrl+D` — Exit the CLI. Follows standard shell behavior, requiring a double-press to exit.
- `Ctrl+J` or `+Enter` — Universal alternatives for inserting newlines that work in all terminals.

> `Shift+Enter` works in iTerm2, Ghostty, Kitty, Warp, and Zed. For tmux users, use `Ctrl+J` instead. See Terminal Setup for configuration options and troubleshooting.

Review

Review changes with `Ctrl+R`. Press `i` to add follow-up instructions. Use `Arrow Up / Arrow Down` to scroll, and `Arrow Left / Arrow Right` to switch files.

Selecting context

Select files and folders to include in context with `@`. Free up space in the context window by running `/compress`.

Cloud Agent handoff

Push your conversation to a Cloud Agent and let it keep running while you're away. Prepend `&` to any message to send it to the cloud. Pick it back up on web or mobile at cursor.com/agents.

```
# Send a task to Cloud Agent mid-conversation
& refactor the auth module and add comprehensive tests
```

CLI worktrees

Pass `--worktree` to run the agent in a new Git worktree instead of editing your current checkout directly. Cursor creates these checkouts under `~/.cursor/worktrees`, alongside worktrees created from the editor.

Cursor cleans up CLI worktrees with the same retention rules it uses for editor worktrees. For cleanup settings and limits, see How are old worktrees cleaned up?.

Combine `--workspace <path>` when you need an explicit repository root. Otherwise the CLI uses the current working directory. `--worktree` only changes where the agent makes file edits inside that project.

```
# Create a temporary worktree from the current repository
agent --worktree "upgrade the test runner and fix any broken snapshots"

# Target a repository from anywhere, but keep the changes isolated
agent --workspace ~/src/my-app --worktree "fix the flaky auth test and open a
```

History

Continue from an existing thread with `--resume [thread id]` to load prior context.

To resume the most recent conversation, use `agent resume`, `--continue`, or the `/resume` slash command.

You can also run `agent ls` to see a list of previous conversations.

Command approval

Before running terminal commands, CLI will ask you to approve (`y`) or reject (`n`) execution.

Non-interactive mode

Use `-p` or `--print` to run Agent in non-interactive mode. This will print the response to the console.

With non-interactive mode, you can invoke Agent in a non-interactive way. This allows you to integrate it in scripts, CI pipelines, etc.

You can combine this with `--output-format` to control how the output is formatted. For example, use `--output-format json` for structured output that's easier to parse in scripts, or `--output-format text` for plain text output of the agent's final response.

Cursor has full write access in non-interactive mode.